PennyPal — Viva & Presentation Script

TechWiz 7: The World Tech Championship
Center: Aptech North Nazimabad Center | **Team:** TechBeast
Theme: NextGen BudgetBee | **Category:** Multi-Platform App Computing
Project: PennyPal — Make Every Penny Count

Executive Summary & Setup

- Target Duration: 5 to 7 Minutes (followed by Judge Q&A)
- Representing Center: Aptech North Nazimabad Center
- Representing Team: TechBeast
- Student Login: ammad.anees99@gmail.com / Ammad1234
- Admin Login: admin@pennypal.com / Admin@1234
- Core Architecture: Flutter 3.24+, Riverpod 2.0 (StreamProviders & Notifiers), Firebase Firestore (Offline-First Cache), Google Gemini Vision OCR
- Value-Added Features Beyond SRS: Split Bill Engine, Borrow & Lend (IOU) Tracker, Subscriptions Manager

---

Section 1: Self & Team Introduction (30 Seconds)

Spoken Script:

"Good morning, everyone! First of all, thank you so much for giving us this opportunity to present our project.

My name is **Muhammad Ammad Anees**, and I'm the team lead of **Team TechBeast** representing Aptech North Nazimabad Center in TechWiz 7 under the Multi-Platform App Computing category.

Before we get started, let me quickly introduce my team members:

- **Muhammad Hadi**, who worked on the UI design,
- **Muhammad Sharjeel**, who worked on the admin panel, and
- **Abdul Qadir**, who handled our documentation.

Together, we developed **PennyPal** — an intelligent personal finance companion for students built on one simple motto: **'Make Every Penny Count.'**"

Speaker Tips:

- Start with a natural smile and a confident greeting.
- Keep your tone conversational and energetic, not robotic.
- Emphasize the app motto at the end: "*Make Every Penny Count.*"

Section 2: Why We Chose PennyPal (30 Seconds)

Spoken Script:

"Now, before we dive into the live app, I'd like to briefly share why our team chose this specific project.

When we received the three project tracks in TechWiz, PennyPal stood out to us immediately—because managing money is a challenge we face every single day as students.

Most personal finance apps in the market are designed for working professionals. They have paid subscriptions, complicated accounting features, and they require a continuous internet connection.

On a campus, students manage small daily expenses in cash and digital wallets, often in classrooms or basements with poor connectivity.

We wanted to build a practical, reliable companion tailored to student life—one that is fast, easy to use, and works completely offline."

Speaker Tips:

- Use the opening line ("*Now, before we dive into the live app...*") as a natural bridge so your speech flows effortlessly from your team intro.
- Speak naturally and clearly. Showing real student empathy helps the judges immediately connect with your purpose.

Section 3: Technology Stack & Architecture (45 Seconds)

Spoken Script:

"To turn this vision into a reality that students could genuinely rely on anywhere on campus, we needed an architecture that is rock-solid and completely offline-ready.

We developed PennyPal using **Flutter** to deliver a responsive, 60 FPS experience across mobile, tablet, and web from a single codebase.

For state management, we implemented **Riverpod 2.0**, which gives us compile-time safety and handles real-time streams without unnecessary widget rebuilds.

For our database, we used **Cloud Firestore** with offline cache persistence enabled, so students can log transactions anytime, even without an active internet connection.

We also integrated **Google Gemini Vision AI** for receipt scanning, backed by an offline regex parser to handle network dropouts.

Now, having seen the foundation under the hood, let's jump straight into the application itself. Let me share my screen and walk you through the live demo."

Speaker Tips:

- Deliver this smoothly and calmly. Notice how the opening and closing lines naturally bridge from 'Why we built it' directly into the live screen share.

Section 4: Live Demonstration Walkthrough (3 to 4 Minutes)

(Share your screen. Speak each line naturally as your hands perform the exact action on screen.)

Step A: Secure Login & Route Guards

👉 **[Live Action]:** Launch PennyPal. Type `ammad.anees99@gmail.com` and password `Ammad1234`. Tap **Sign In**.

Spoken Script:

"We begin right here at the login screen. The application uses GoRouter with declarative route guards to keep user sessions secure. When I enter our student credentials and tap Sign In, the app verifies the account against Firebase and routes us directly into the student dashboard."

Step B: The Student Dashboard & Real-Time Balance

👉 **[Live Action]:** Point with mouse or tap on Income, Expense, Net Balance cards, and the central Budget Ring.

Spoken Script:

"And the first thing a student lands on is the Dashboard. Right at the top, they get an immediate summary of their finances: Total Income, Monthly Expenses, and Net Balance, combined across cash in pocket, bank cards, and mobile wallets like Nayapay or EasyPaisa.

Right here in the center, this animated ring shows exactly how much of this month's budget has been consumed so far, calculated in real time by our core finance engine."

Step C: Calendar-Month Budgeting & 1-Tap Quick Copy

👉 **[Live Action]:** Tap the **Budget** tab. Show category caps. Tap the forward arrow `>` to next month, highlighting the **1-Tap Quick Copy** banner.

Spoken Script:

"Now, seeing your balance is one thing, but preventing overspending is what truly matters. So let's head over to the Budget section.

Budgets in PennyPal are organized strictly by calendar month, matching how student allowances and hostel rent operate. Students can set spending caps for individual categories.

And to save students from tedious data entry at the start of every month, we built a **1-Tap Quick Copy** feature. Instead of typing every category limit again, a single tap copies the previous month's budget automatically."

Step D: Quick Expense Logging & AI Receipt Scanner

👉 **[Live Action]:** Tap **Add Expense (+)**. Show the 8 categories. Tap **Attach Receipt** to showcase camera/gallery scan.

Spoken Script:

"Once a budget is in place, the student needs to log their daily spending. Adding an expense takes just a few seconds across all 8 standard categories, like Food, Transport, and Education.

In addition to manual entry, students can scan paper receipts. Using Google Gemini Vision AI, the app scans the photo, extracts the merchant name, transaction date, and total amount, and pre-fills the form automatically."

Step E: The Live Offline Engine (Evaluation Highlight!)

👉 **[Live Action]:**

1. Pull down quick settings and turn **ON Airplane Mode** (Zero Wi-Fi, Zero mobile data).
2. Log a $15 expense for 'Campus Lunch'. Tap **Save**. Show it appearing instantly in the list.
3. Turn **OFF Airplane Mode**. Point out silent cloud synchronization.

Spoken Script:

"Now, here is a real-world scenario every student encounters on campus: what happens when your Wi-Fi drops or you're in a basement classroom with zero signal? Let me demonstrate how PennyPal handles this.

I will turn on **Airplane Mode** to simulate having zero internet connection.

When I log a 15-dollar expense for lunch, notice that it commits immediately to local storage and updates the dashboard without any loading spinner or lag.

And the moment I turn Airplane Mode back off, Firebase automatically detects the restored connection and synchronizes the transaction to the cloud in the background."

Step F: Savings Goals & Milestone Motivation

👉 **[Live Action]:** Tap **Savings** tab. Open an active goal (e.g. 'New Laptop'). Tap **Contribute**, enter `$20`, and tap Save.

Spoken Script:

"Keeping daily expenses in check is essential, but building long-term financial health requires saving. That brings us to our Savings Goals.

Here, students can set dedicated targets—such as saving for a laptop, a certification, or emergency cash—with specific deadlines. When we add a contribution, the progress bar updates, the app recalculates the remaining timeline, and provides milestone encouragement to help students stay disciplined."

Step G: Student-Centric Features

👉 **[Live Action]:** From the dashboard, open **Split Bill**, then show **Borrow & Lend**, then **Subscriptions**.

Spoken Script:

"Now, while testing the app with our classmates, we realized that student life involves social moments that traditional banking apps completely ignore. That's why we engineered three practical student utilities right on the dashboard:

First, **Split Bill**—which calculates each person's exact share when dining out with classmates and lets you copy a payment reminder to WhatsApp in one tap.

Second, **Borrow and Lend**—to keep a clear, transparent record of small informal peer debts so friendships don't get awkward.

And third, **Subscriptions Manager**—to track recurring monthly services like software or gym passes so they don't quietly drain your allowance."

Step H: Learning Hub & AI Financial Assistant

👉 **[Live Action]:** Tap **Learning**, open a lesson card ('Needs vs Wants'). Then open **Assistant** tab and ask a quick spending question.

Spoken Script:

"Beyond daily tracking, we also wanted to empower students with financial knowledge. That's why our **Learning Hub** offers bite-sized lessons on topics like needs versus wants and emergency funds.

And if a student is ever unsure about their spending decisions, they can consult our **AI Assistant** for personalized advice on budgeting and saving."

Step I: Preconfigured Administrator Portal

👉 **[Live Action]:** Tap Profile -> Logout. Log in as `admin@pennypal.com` with password `Admin@1234`. Show the Admin Dashboard.

Spoken Script:

"Everything we've seen so far is from the student's perspective. But a campus platform also needs administrative oversight. So let's quickly switch over to the Administrator Portal.

When an administrator signs in, they access a dedicated administrative dashboard to monitor registered students, inspect system analytics, resolve support tickets, and publish new educational lessons through the built-in CMS."

Section 5: Closing Statement & Handover (20 Seconds)

Spoken Script:

"To bring everything together—PennyPal was designed from day one to be more than just a digital ledger. It is a dependable, offline-ready companion that genuinely helps students take control of their finances and build healthy habits for life.

On behalf of team **TechBeast** and **Aptech North Nazimabad Center**, thank you very much for your time and guidance today. We would be delighted to answer any questions you have!"

Speaker Tips:

- Close with confidence, maintain eye contact, and leave your screen showing the Student Dashboard as a visual reference for the judges.

Section 6: Top 20 Technical Judge Questions & Model Answers

Q1: "Why did you implement Split Bill, Debt Tracker, and Subscriptions when they were not in the SRS baseline?"

Answer: "SRS Section 1.9 explicitly encourages applying creativity to improve the cross-platform application. Our research at Aptech showed that split canteen meals, forgotten peer debts, and recurring subscriptions are the three largest causes of student financial leakage. Adding these completed the real-world student financial workflow."

Q2: "How does Riverpod manage state compared to Provider or Bloc?"

Answer: "Riverpod is a complete rewrite of Provider that operates outside Flutter's widget tree. It provides compile-time safety with no runtime ProviderNotFoundExceptions. We use StreamProviders for real-time Firestore synchronization and StateNotifiers for form and filter state, allowing selective widget rebuilds without unnecessary re-renders."

Q3: "What is the exact difference between ref.watch() and ref.read() in your code?"

Answer: "We use ref.watch() inside build() methods to subscribe widgets to reactive state changes so the UI rebuilds when data updates. We use ref.read() strictly inside event handlers and callbacks, such as button onTap functions, to read a provider once or trigger a method without creating an unwanted rebuild subscription."

Q4: "What happens if a student sets a budget mid-month on October 20th?"

Answer: "The budget document is keyed deterministically by month as YYYY-MM (e.g., 2026-10). All transactions recorded from October 1st to 20th are immediately aggregated against the new budget limit, updating remaining funds and triggering overspending warnings if past spending already exceeded the limit."

Q5: "How does receipt OCR work if Gemini API is offline or times out?"

Answer: "We engineered a Dual-Engine architecture in ReceiptOcrService. When an image is scanned, it queries the Gemini Vision API. If the call times out after 12 seconds or fails with a network exception, it seamlessly cascades to our local heuristic regex parser, ensuring zero downtime for the user."

Q6: "How is student data secured in Firestore?"

Answer: "We implement multi-tenant security rules in firestore.rules. Every document path is scoped under users/{userId}. Our rules enforce that request.auth.uid matches the document userId. No student can read or write another student's ledger. Furthermore, sensitive CVV and full card numbers are never stored."

Q7: "How do you prevent negative balances or warn students of overdrafts?"

Answer: "Our FinanceCalculator computes live running balances per payment mode. If an expense exceeds the balance in a specific mode, such as spending $50 from a cash wallet holding $20, the app flags an overdrawn warning and updates the Financial Health Index into the 'Needs Attention' tier."

Q8: "How is the Financial Health Score calculated?"

Answer: "It is a 100-point algorithm divided into three pillars:

1. Budget Discipline (40 points): Compares actual spend against expected burn rate using a dynamic time-decay pace ratio (daysElapsed / totalDays).
2. Savings Margin (30 points): Evaluates whether savings exceed 25% of income.
3. Habit Consistency (30 points): Rewards daily transaction logging regularity and active savings goals."

Q9: "How does offline synchronization prevent data corruption?"

Answer: "Firestore provides client-side offline persistence. Local writes receive optimistic timestamps and a hasPendingWrites flag. When the connection re-establishes, Firestore synchronizes changes via atomic transactions. In the event of conflicting edits, Firestore's last-write-wins resolution guarantees document integrity."

Q10: "Why limit payment cards to two?"

Answer: "This is an intentional behavioral finance design choice. Research indicates that students juggling multiple credit and debit cards face higher rates of impulse spending and missed payments. Restricting to two cards encourages disciplined primary and emergency card usage and keeps the UI clean."

Q11: "How does PennyPal handle push and in-app notifications?"

Answer: "We use flutter_local_notifications paired with domain notification triggers. When a user logs an expense that crosses 80% or 100% of their monthly category budget, or when a subscription renewal or debt due date is within 48 hours, the app dispatches an immediate local notification."

Q12: "How do you optimize image handling for receipt attachments to prevent memory leaks?"

Answer: "Receipt photos are captured via image_picker and compressed before uploading. On the UI side, images are loaded using memory caching with fixed decode dimensions (cacheWidth and cacheHeight), preventing high-resolution camera images from consuming excessive device RAM."

Q13: "How does the app scale if a student logs thousands of transactions over four years?"

Answer: "Our queries are partitioned by calendar month and indexed with compound Firestore indexes on (userId, date). The app only queries the active month's transactions for the dashboard, using pagination and date-range limits on the Activity screen so data payloads remain tiny regardless of total history size."

Q14: "What automated tests did you write to ensure financial math accuracy?"

Answer: "We wrote automated Dart unit tests covering our domain calculation engines: FinanceCalculator, ReportCalculator, the budget pace ratio formula, and debt split rounding. Because our domain layer contains pure Dart models with zero Flutter UI dependencies, tests execute in milliseconds with 100% determinism."

Q15: "How does the app protect against SQL or NoSQL injection?"

Answer: "PennyPal uses the official Cloud Firestore SDK, which structures all operations as parameterized objects and typed document references rather than raw query strings. User inputs in text fields are strictly data payload values, making injection attacks impossible."

Q16: "What happens if a student changes their device clock to bypass a budget or goal deadline?"

Answer: "While local dates assist with offline entry, all cloud persistence uses Firestore's server-assigned timestamps (FieldValue.serverTimestamp()). When data synchronizes with the backend, cloud validation guarantees audit accuracy regardless of client clock tampering."

Q17: "How did you ensure responsive UI across phones, foldables, and tablets?"

Answer: "We built an AdaptiveScaffold and custom responsive breakpoints. On standard mobile screens, navigation uses a bottom navigation bar. On tablets and desktop web browsers, the layout automatically transforms into a persistent navigation rail with multi-column grid cards, maximizing screen real estate."

Q18: "Can students export reports for parents or university scholarship audits?"

Answer: "Yes. From the Reports module, students can generate comprehensive monthly statements and export their full ledger to CSV or formatted print-ready PDF files using Flutter's printing and pdf packages."

Q19: "How do you support international currencies for students studying abroad?"

Answer: "Our CurrencyService allows students to select their local currency symbol (such as PKR, USD, EUR, GBP, or INR). All financial formatting in the UI passes through our centralized CurrencyFormatter, ensuring seamless formatting across all screens."

Q20: "What is your roadmap for monetization or scaling PennyPal beyond TechWiz?"

Answer: "Our roadmap follows a B2B2C model: licensing PennyPal directly to universities and colleges as an official campus wellness tool, integrating student meal cards, and offering curated affiliate discounts on educational supplies and textbooks."

Section 7: Professional Viva Delivery Guidelines

Clarification Phrases (If you need the judge to repeat):

- If audio is unclear: "Pardon me sir/ma'am, could you please repeat the question?"
- If the question is fast: "Could you please elaborate on that point, sir?"
- To verify the question before answering: "Are you inquiring about our offline synchronization mechanism, or our Riverpod state flow?"
- To demonstrate proof on screen: "Yes sir, let me demonstrate that live on screen right now."

Key Presentation Rules:

1. Speak in solid, clear sentences. Never rush.
2. When answering technical questions, name the specific file or class (e.g., FinanceCalculator, ReceiptOcrService, StreamProvider).
3. Always keep your emulator or device running with live test data loaded before the meeting starts.

Good luck Team PennyPal (Aptech North Nazimabad Center)